

CAD-Recode: Reverse Engineering CAD Code from Point Clouds

Danila Rukhovich¹

danila.rukhovich@uni.lu

Elona Dupont¹

elona.dupont@uni.lu

Dimitrios Mallis¹

dimitrios.mallis@uni.lu

Kseniya Cherenkova^{1,2}

kseniya.cherenkova@uni.lu

Anis Kacem¹

anis.kacem@uni.lu

Djamila Aouada¹

djamila.aouada@uni.lu

¹SnT, University of Luxembourg ²Artec3D, Luxembourg

Abstract

Computer-Aided Design (CAD) models are typically constructed by sequentially drawing parametric sketches and applying CAD operations to obtain a 3D model. The problem of 3D CAD reverse engineering consists of reconstructing the sketch and CAD operation sequences from 3D representations such as point clouds. In this paper, we address this challenge through novel contributions across three levels: CAD sequence representation, network design, and training dataset. In particular, we represent CAD sketch-extrude sequences as Python code. The proposed **CAD-Recode** translates a point cloud into Python code that, when executed, reconstructs the CAD model. Taking advantage of the exposure of pre-trained Large Language Models (LLMs) to Python code, we leverage a relatively small LLM as a decoder for CAD-Recode and combine it with a lightweight point cloud projector. CAD-Recode is trained on a procedurally generated dataset of one million CAD sequences. CAD-Recode significantly outperforms existing methods across the DeepCAD, Fusion360 and real-world CC3D datasets. Furthermore, we show that our CAD Python code output is interpretable by off-the-shelf LLMs, enabling CAD editing and CAD-specific question answering from point clouds.

1. Introduction

Computer-Aided Design (CAD) modeling is the standard approach for designing manufactured objects, ranging from furniture to mechanical components [3, 30]. However, creating a 3D CAD model is a time-consuming task that demands specialized expertise, as the model must not only capture the object’s shape but also maintain its functional requirements—commonly referred to as the *design intent* [4, 27]. To streamline this process, *3D CAD reverse engineering* aims at generating CAD models directly from 3D scanned objects, offering a faster and more accessible

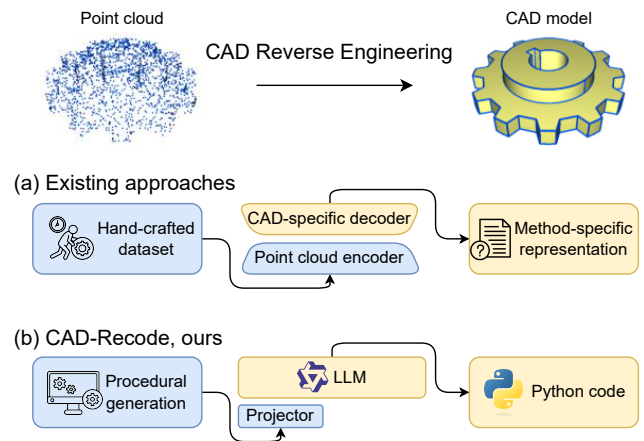


Figure 1. 3D CAD reverse engineering is a process of converting a point cloud into a CAD model (top). Existing methods are constrained by the use of method-specific CAD representations and limited hand-crafted training datasets (a). On the contrary, CAD-Recode employs a pre-trained LLM with a lightweight projector that translates point clouds into executable Python code and is trained on a procedurally generated dataset (b).

pathway to CAD creation [17].

Automated 3D CAD reverse engineering has a long history in the fields of computer vision and graphics [13, 42], with goals evolving alongside advancements in the field. These objectives have progressed from identifying CAD parts in 3D point clouds [42] to predicting the sequence of steps a designer may take to recreate a 3D scanned object in CAD software [21, 34]. This latter goal is particularly appealing, as it aims not only to produce a final CAD parametric model but also to capture the design steps behind it, enabling further editing within CAD software [21, 43]. In CAD software, designers typically construct their CAD model as feature-based design sequences, where a sequence of 2D sketches is transformed into 3D objects via opera-

tions such as extrusion and revolution [45, 48]. Following the release of large CAD datasets [7, 22, 45], recent works have focused on learning feature-based CAD sequences from input point clouds, specifically as *sketch-extrude* sequences [12, 21, 28, 33, 38, 43, 46]. As depicted in Figure 1(a), although varying in methodology, these approaches share a common pipeline: (1) crafting a CAD sketch-extrude sequence representation, (2) converting raw CAD data [22, 45] into this format, and (3) training dedicated neural networks to output these representations from input point clouds.

Despite recent advances in feature-based CAD reverse engineering, key limitations constrain the broader applicability of existing approaches. Firstly, existing methods often use customized CAD representations, such as custom CAD languages [12, 21, 34, 46] or simplified extrusion shapes [28, 38, 43], to facilitate model training. These representations are difficult to interpret, require post-processing to be compatible with CAD tools, and restrict design capabilities to basic operations. Secondly, these approaches typically rely on designing networks that output language-like CAD representations [12, 21] and training them from scratch. This requires the networks to learn not only the geometry of the point clouds, but also the syntax of the CAD sequence representation.

In this paper, we pose the following question: *In view of the recent breakthrough performance of Large Language Models (LLMs), how can their advanced language understanding be leveraged for CAD reverse engineering?*

To address this question, we base our approach on three key observations: (1) LLMs can generate valid Python code [36, 53], (2) modern CAD software increasingly supports modeling through Python scripting [10], and (3) recent efforts have shown that LLMs can be fine-tuned to process point clouds [47, 59]. As shown in Figure 1(b), we propose *CAD-Recode*, a solution for CAD reverse engineering by fine-tuning an LLM to map input point clouds into CAD sketch-extrude sequences represented as Python code. In particular, instead of crafting a CAD representation, we base our representation on the existing Python CadQuery library [10]. This code-based representation is not only interpretable but also inherently allows for incorporating modular CAD features and design practices such as reusing design elements and abstracting low-level design steps (e.g. 3D box to represent a four-line sketch of a square and its extrusion). To learn the mapping between point clouds and CAD Python code, we fine-tune a pre-trained LLM, Qwen2-1.5B [53], augmented with a lightweight, trainable point cloud projector. To train *CAD-Recode*, a potential approach could be using existing sketch-extrude datasets [45, 46] and converting them to Python CadQuery code. However, these datasets are limited in size and design features included due to the efforts required to con-

vert their original proprietary representation into one that is suitable for learning. As a solution, we propose a *procedurally* generated training dataset composed of one million CAD sketch-extrude sequences as Python CadQuery code. This dataset consists of CadQuery Python scripts generated following predefined heuristics with randomized parameter selection. The execution of each generated script results in a parametric CAD model. Unlike existing CAD datasets, this procedurally generated dataset provides an alternative for learning the mapping between point clouds and CAD sketch-extrude sequences in Python code, with full control over the design features, patterns and dataset size included during training. Our contributions can be summarized to:

- A CAD sketch-extrude sequence representation in Python code using CadQuery [10] for CAD reverse engineering.
- *CAD-Recode*, the first LLM-based CAD reverse engineering model designed to predict CAD Python code from point clouds. The model, consisting of a pre-trained LLM and a point cloud projector is trained end-to-end to generate code that reconstructs the input geometry.
- A one million procedurally generated training dataset of CAD sketch-extrude sequences as CadQuery Python code. This provides precise control over dataset size, features, and design patterns included during training, resulting in significant performance improvement. We will make this dataset publicly accessible.
- Extensive experiments on three publicly available datasets show that *CAD-Recode* achieves substantial improvements over state-of-the-art methods in CAD reverse engineering. Moreover, we show that *CAD-Recode*, when operating on point clouds and generating CAD code, can be integrated with an off-the-shelf LLM to perform CAD Question Answering (CAD-QA) and CAD editing from point clouds.

2. Related Works

LLM, Point Cloud and CAD: Recent works have explored integrating point clouds with LLMs for various tasks, including 3D generation [54, 60], captioning [16, 18, 47], and question answering [6, 19]. These approaches typically employ complex point cloud encoders, either aligning with CLIP embeddings [31, 51, 52, 59, 62] or directly with LLM feature spaces [47]. Such methods require two-stage training: first pre-training the point cloud encoder, then fine-tuning with the LLM through instruction-based prompts.

In parallel, recent works have started exploring LLMs’ capabilities in a range of CAD-related tasks. Reparam-CAD [23] infers shape variations from parametric models and text descriptions, while CADTalk [58] generates semantic descriptions of CAD parts. The works in [1, 2] focus on the generation of CAD models from text using LLMs, and SGP-Benchmark [37] evaluates LLMs’ under-

standing of CAD sketch-extrude sequences using CAD-specific question answering. While Img2CAD [55] attempts CAD reverse engineering from images using GPT-4V [36], it still requires a separate transformer for parameter inference. In contrast, CAD-Recode introduces the first approach for point cloud to CAD reconstruction combining point clouds with the sequence modeling capabilities of pre-trained LLMs.

CAD Reverse Engineering from Point Cloud: CAD reverse engineering aims to reconstruct parametric CAD models from 3D shapes (*e.g.*, meshes or point clouds) in a compatible representation with CAD software. A key challenge lies in the choice of this representation. A line of works attempts to address sub-problems of the CAD reverse engineering task by focusing on parameter estimation for edges and surface primitives [8, 9, 15, 26, 32, 41, 44, 61] or reconstructing B-Rep construction history [11, 24, 45, 48]. In order to obtain a representation that is closer to CAD modelling, several methods [14, 20, 29, 56, 57] use Constructive Solid Geometry (CSG), representing models as compositions of 3D primitives and Boolean operations. While this enables reconstruction, the CSG representation diverges from modern CAD workflows [48].

Recent works have adopted the more CAD-aligned sketch-extrude representation, introduced by DeepCAD [46] for CAD generation [49, 50] or predicting extrusion cylinder [38, 43]. Considering the sequential nature of sketch-extrude operations, methods have explored a template-based approach [25] given a rounded voxel input representation. Furthermore, transformer architectures have been investigated for both autoregressive [21] and non-autoregressive [12, 46] prediction of sketch-extrude sequences from point clouds. The work in [34] combines a lightweight pre-trained language model [39] with a point cloud encoder using a diffusion-based approach. Alternative methods using self-supervised [28] or unsupervised [29] learning still face integration challenges due to their non-standard sketch representations (*e.g.*, signed distance functions). In contrast to these approaches that require full parameter learning of specialized networks for both CAD geometry and representation syntax, we leverage pre-trained LLMs that have been exposed to programming patterns through large-scale training on code repositories. Our method outputs Python code using the CadQuery library [10] that is directly executable and can easily be interpreted. Additionally, we address the data limitation through automated generation of a large-scale training dataset, enabling full control over design features included during training.

3. CAD Representation as Code

Modern feature-based CAD modeling relies on se-

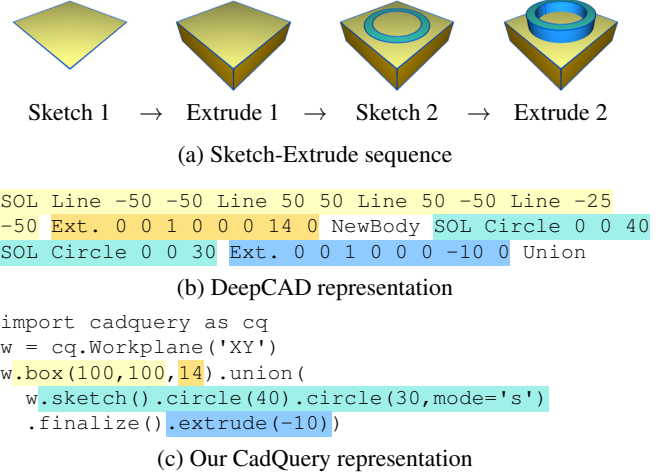


Figure 2. Sketch-extrude sequence (top) in DeepCAD representation (middle) and our CadQuery code (bottom).

quences of 2D sketches and operations to create 3D models. Designers first draw geometric primitives (lines, arcs, circles) on a selected plane, then apply operations like extrusion or revolution to generate 3D geometry [48]. As depicted in Figure 2(a), we focus on sketch-extrusion sequences, a fundamental CAD modeling pattern widely adopted in previous works [21, 46, 49]. Below, we present our CAD representation, highlighting its advantages over existing language-like encodings, and describe our procedurally generated training data.

3.1. CadQuery Code

Recent approaches in CAD language modeling [12, 21, 33, 46, 49] encode sketch-extrude sequences as numerical vectors representing features and their parameters as shown in Figure 2(b). However, this representation constrains the modeling to specific CAD practices, lacks interpretability, and requires post-processing for CAD kernel compatibility. We propose using CadQuery [10] Python code to represent sketch-extrude sequences for CAD reverse engineering, offering the following advantages:

Modularity of CAD Features and Design Practices: Existing language-based CAD reverse engineering methods rely on custom representations of low-level geometric features (lines, arcs, circles) for sketch construction [40, 46]. This approach inherently limits the range of implementable features and design practices. In contrast, CadQuery provides comprehensive built-in CAD functionality, encompassing both low-level features and higher-level geometries like cylinders and boxes as shown in Figure 2(c). Furthermore, its programmatic nature enables variable reuse and code modularity. This allows reusing common design features or practices across models, as illustrated by the shared center coordinates across two circles in Figure 2 (c). This

representation naturally accommodates diverse CAD features and design practices without requiring complex custom encodings or post-processing steps.

Interpretability and LLM Compatibility: The proposed representation, based on Python and CadQuery syntax, presents an alternative to abstract numerical encodings with improved interpretability. Its code-based format facilitates model editing both programmatically and through CAD software. Importantly, this representation aligns with pre-trained LLMs’ demonstrated proficiency in Python code generation and manipulation. Indeed, state-of-the-art proprietary LLMs like GPT-4 [36] achieve over 90% accuracy on the Python code HumanEval benchmark [5], while even lightweight open-source models such as Qwen2-1.5B [53] show promising code generation capabilities. Hence, this code representation facilitates fine-tuning of pre-trained LLMs for the specific task of reverse engineering point clouds into CAD Python code and opens the doors for new capabilities with off-the-shelf LLMs.

3.2. Procedurally Generated Training Dataset

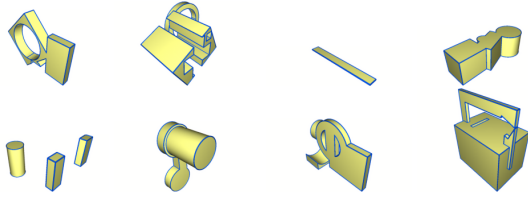


Figure 3. Examples of procedurally generated CAD models.

The training of current CAD sketch-extrude reverse engineering methods [12, 21, 33, 34, 46] predominantly rely on datasets collected from CAD model repositories [7, 22, 45]. Considerable efforts are required to parse the CAD models from their original proprietary representations to a suitable one for deep learning [45, 46]. As a result, existing datasets are restricted not only in scale, but also in control over the design features and patterns included in training.

To address these limitations, we propose to procedurally generate a training dataset of one million CAD models in the form of sketch-extrude sequences written in Python CadQuery [10] code. Our proposed pipeline randomly generates sketch and CAD operation parameters guided by topological and geometrical heuristics to ensure control over the amount of generated models and the features in the generated codes. The algorithm outlining the steps of this generation pipeline is provided in the supplementary materials along with further statistical analysis of the generated dataset. The modularity of CAD features is incorporated by utilizing both low-level primitives (*i.e.* circles, lines, and arcs) and their abstractions (*i.e.* boxes, cylinders, and rectangles) as well as reusing design elements within

the generated sequences. In this work, we focus on some aspects of modularity (*i.e.*, reusing point coordinates, extrusion planes, and abstracting basic shapes such as boxes and cylinders). Further modularity features (*e.g.*, reusing functions corresponding to arbitrary CAD parts, additional CAD operations) can also be integrated in the future. Note that although our generated dataset does not include sequences from human-designed CAD models, it offers significant control over the features and design patterns to which the network is exposed during training. Examples of generated CAD models are shown in Figure 3.

4. CAD-Recode

Building on the introduced CAD code representation and generated training dataset outlined in the previous section, this section introduces CAD-Recode, our proposed model for predicting CAD sketch-extrude sequences as code from input 3D point clouds. We formalize the problem of CAD code prediction, describe the architecture of CAD-Recode, and detail its training and inference processes.

4.1. Problem Statement

Let us denote the set of all possible code strings as Σ^* , where each code string is composed of elements from the finite set of alphanumeric characters and operators in the programming language Σ . Let $\phi_{\text{syn}} : \Sigma^* \rightarrow \{\text{True}, \text{False}\}$ represent the syntactic validation function for Python programming rules (*e.g.*, variable declarations, expression syntax, and statement structure), and $\phi_{\text{cad}} : \Sigma^* \rightarrow \{\text{True}, \text{False}\}$ denote the validation function for CAD-specific rules. The latter includes the syntactic validity of the code w.r.t. to the CAD library, *i.e.* CadQuery [10], and the geometric validity of the reconstructed model from the code (*e.g.*, an extrusion can only be applied on a closed loop of 2D primitives, a circle radius cannot be negative). An executable valid CAD code can be formally described by a code string $C \in \mathcal{C}$, where

$$\mathcal{C} = \{w \in \Sigma^* \mid \phi_{\text{syn}}(w) \wedge \phi_{\text{cad}}(w)\},$$

represents the set of all valid CAD codes. This formulation ensures that any code string w in $\mathcal{C}$ satisfies both the syntactic requirements of Python (ϕ_{syn}) and the CAD code validation rules (ϕ_{cad}). Let $\mathbf{P} = \{\mathbf{p}_i\}_{i=1}^n \in \mathbb{R}^{n \times 3}$ denote an input point cloud, where each point $\mathbf{p}_i \in \mathbb{R}^3$ represents 3D Cartesian coordinates. The objective of CAD-Recode is to learn a mapping

$$\Psi : \mathbb{R}^{n \times 3} \rightarrow \mathcal{C}, \quad C = \Psi(\mathbf{P}),$$

that maps the input point cloud to a valid CAD code $C \in \mathcal{C}$ such that the code, when executed, produces a CAD model geometrically approximating the input point cloud $\mathbf{P}$. Note that the CAD code execution results in a

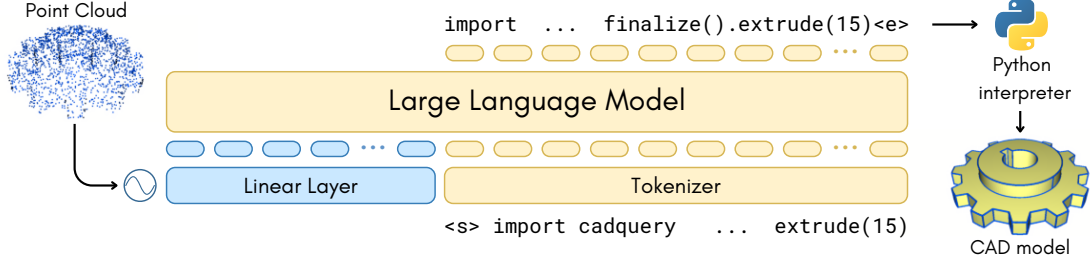


Figure 4. Overview of CAD-Recode. The pipeline comprises two parts: (1) a point cloud projector (marked blue) (2) a fine-tuned pre-trained LLM (yellow). An input point cloud is processed using (1), and outputs are then passed to an LLM (2), which predicts a CAD sketch-extrude sequence in the form of executable Python code.

Boundary-Representation (B-Rep) [24]. Unlike meshes or point clouds, B-Rep is a parametric representation of the CAD model’s geometry, enabling seamless integration into modern CAD software and allowing for further modifications. The goal of CAD-Recode is to infer the CAD code describing the design steps of the CAD model, that when executed results in a B-Rep.

4.2. Proposed Model Architecture

CAD-Recode builds on pre-trained LLMs and their prior exposure to Python code, augmenting these with point cloud processing capabilities and CAD-specific Python code knowledge. As shown in Figure 4, its architecture consists of two components: (1) a point cloud projector mapping the 3D point cloud into learnable tokens, and (2) a pre-trained LLM-based autoregressive CAD code decoder.

Point Cloud Projection Module: CAD-Recode introduces a lightweight projection module Ψ_p that directly maps a dense point cloud $\mathbf{P} \in \mathbb{R}^{n \times d_p}$, where $d_p = 3$ corresponds to the dimension of point coordinates, into a sequence of $n_p \ll n$ query tokens $\mathbf{Q}_p = [\mathbf{q}_p^1, \dots, \mathbf{q}_p^{n_p}] \in \mathbb{R}^{n_p \times d_q}$, of embedding dimension d_q . The point cloud projector, trained in an end-to-end manner with the CAD code decoder module, consists of three simple components: (1) furthest point sampling to downsample the input point clouds to n_p points, (2) Fourier positional encoding [60] of coordinates, and (3) a linear layer projecting the encoded coordinates into $\mathbf{Q}_p$.

LLM as CAD Code Decoder: Our CAD code decoder, denoted as Ψ_{LLM} , adapts a pre-trained LLM for the specific task of CAD code generation. We leverage the Qwen2-1.5B model [53] as our LLM backbone, chosen for its balanced trade-off between model capacity and computational requirements. The decoder’s input consists of point query tokens $\mathbf{Q}_p$ from the point cloud projector, augmented with n_t code tokens $\mathbf{Q}_t \in \mathbb{R}^{n_t \times d_q}$ obtained by tokenizing the input code as in [53]. The complete input sequence is denoted as $[\mathbf{Q}_p; \mathbf{Q}_t] \in \mathbb{R}^{(n_p+n_t) \times d_q}$, where semicolon indicates concatenation along the sequence dimension. The LLM de-

coder generates the CAD code sequence through next-token prediction. As in [53], each predicted token is mapped to a symbol from the vocabulary Σ , which includes alphanumeric characters and operators.

Overall, CAD-Recode repurposes the LLM’s sequence modeling capabilities for the specialized task of translating geometric point clouds into executable CAD code.

4.3. Training and Inference Details

Training Strategy: Our training process consists of a single stage. The model operates on query tokens of dimension $d_q = 1536$ and processes input point clouds downsampled to $n_p = 256$ points. Gaussian noise with mean zero and standard deviation of 0.01 is added to the coordinates of the input points with a probability of 0.5 per model. The network is trained on the procedurally generated CAD codes, hence exposed to the CAD features and design practices that were included in the algorithm. The training objective minimizes the Negative Log-Likelihood (NLL) of the target CAD code sequence, using special tokens (`<s>` and `<e>`) to demarcate sequence boundaries. The point cloud projector Ψ_p learns geometric features from scratch, while the pre-trained decoder Ψ_{LLM} is fine-tuned for CAD code generation.

Inference Strategy: At inference time, the point cloud projector Ψ_p processes the input point cloud $\mathbf{P}$ to generate query tokens $\mathbf{Q}_p$, which are then fed to the decoder along with the start token `<s>`. The model autoregressively generates CAD code tokens until producing a complete code sequence C ending with token `<e>`. Following [21], we employ a test-time sampling approach where we generate ten distinct CAD code candidates, each from a different sampling of the input point cloud. For each candidate, we sample points from the predicted CAD model and compute the Chamfer distance w.r.t. the input point cloud. The candidate with the minimum Chamfer distance is selected as the final output. This verification step effectively favors executable CAD code solutions that are geometrically consistent w.r.t. the input point cloud.

Method	Train Dataset		DeepCAD Test Set				Fusion360 Test Set			
	Name	Size	Mean CD↓	Med. CD↓	IoU↑	IR ↓	Mean CD↓	Med. CD↓	IoU↑	IR↓
DeepCAD [46]	DeepCAD	160k	42.5	9.64	46.7	7.1	330	89.2	39.9	25.2
PrismCAD [25]	DeepCAD	127k	–	4.28	72.1	16.2	–	4.75	65.3	18.0
Point2Cyl [43]	DeepCAD	35k	–	4.27	73.8	3.9	–	4.18	67.5	3.2
HNC-CAD [50]	DeepCAD	125k	–	8.64	65.3	5.6	–	36.8	63.5	7.3
MultiCAD [33]	DeepCAD	160k	–	8.09	–	11.5	–	42.2	–	16.5
TransCAD [12]	DeepCAD	140k	32.3	4.51	65.5	1.1	78.6	33.4	60.2	2.4
CAD-Diffuser [34]	DeepCAD	160k	–	3.02	74.3	1.5	–	3.85	63.2	1.7
CAD-SIGNet [21]	DeepCAD	160k	3.43	0.28	77.6	0.9	7.37	0.48	65.6	1.6
CAD-Recode	DeepCAD	160k	0.89	0.20	86.2	0.0	1.77	0.30	75.6	0.0
CAD-Recode	Ours	1M	0.30	0.16	92.0	0.4	0.35	0.15	87.8	0.5

Table 1. Comparison of CAD reverse engineering methods on DeepCAD and Fusion360 datasets. Our CAD-Recode trained on the 160 k DeepCAD dataset demonstrates an improvement over existing state-of-the-art methods both in terms of geometric fidelity and validity of the generated sketch-extrude sequences. Our procedurally generated dataset provides a significant boost in the prediction quality.

5. Experiments

In order to validate the effectiveness of CAD-Recode, we conduct a series of experiments across two different scenarios. The first scenario focuses on the reverse engineering task, where the goal is to reconstruct a CAD sketch-extrude sequence in Python code from a given input point cloud. The second assesses the interpretability and editability of the generated CAD code with a proprietary LLM [36].

5.1. Reverse Engineering

Experimental Setup: CAD-Recode is evaluated on three test datasets: DeepCAD [46] (8046 models), Fusion360 [45] (1725 models), and the real-world CC3D [35] (2973 models). The point clouds are obtained by sampling points on the meshes for DeepCAD and Fusion360. The CC3D dataset provides a real-world scenario with input point clouds sampled from actual 3D scans of CAD models containing surface noise, smoothed edges and missing parts (see supplementary materials for more details). Implementation details are provided in the supplementary.

Metrics: To evaluate the quality of the predicted CAD sketch-extrude sequences, we use three metrics: Chamfer Distance (CD) [21], Intersection over Union (IoU) [34], and Invalidity Ratio (IR) [46]. We report both mean and median CD values computed using 8192 points to assess geometric accuracy. Reported CD values have been multiplied by 10^3 . The IoU is computed from the resulting CAD model meshes and expressed as a percentage. The IR indicates the percentage of generated sequences that fail to produce a valid CAD model.

Results & Analysis: Table 1 presents results on the test sets of DeepCAD and Fusion360 datasets, where CAD-Recode establishes new state-of-the-art performance across all metrics. Note that the results of state-

of-the-art methods in Table 1 are borrowed from [34], except for CAD-SIGNet [21], MultiCAD [33], TransCAD [12], and DeepCAD [46] which were taken from [21] and [12]. First, we convert the DeepCAD dataset (160 k models) to CadQuery Python code and use it to train CAD-Recode (results are in row before last of Table 1). When trained on DeepCAD dataset as existing methods, CAD-Recode outperforms them in almost all metrics. These results showcase the effectiveness of CAD-Recode and the proposed CAD code representation.

Training on 1 M generated samples results in substantial improvements in CD and IoU metrics while maintaining a negligibly low invalidity ratio (last row of Table 1), reflecting significantly better geometric fidelity in the predicted CAD models. CAD-Recode demonstrates a ten-fold improvement in mean CD and an increase of IoU by over 10% on both DeepCAD and Fusion360 datasets compared to the existing best methods. These results confirm that our large-scale procedurally generated training dataset provides substantial benefits.

As illustrated in Figure 5, this translates to consistent reconstruction quality, where CAD-Recode reliably produces CAD models that accurately capture the geometry from the input point cloud. In contrast, CAD-SIGNet [21] can generate shapes that deviate significantly from the target geometry, further highlighting the advantages of our approach.

Real-world Scenario: In Table 2, we evaluate CAD-Recode on the real-world CC3D dataset, where input point clouds are sampled from 3D scans and contain artifacts such as surface noise, smooth edges, and missing parts. Even under these challenging conditions, our method achieves significant improvements over CAD-SIGNet [21], with a 89% lower median CD and a 30% higher IoU, while maintaining a low IR. From the CC3D qualitative results in Figure 5, CAD-Recode is able to recover

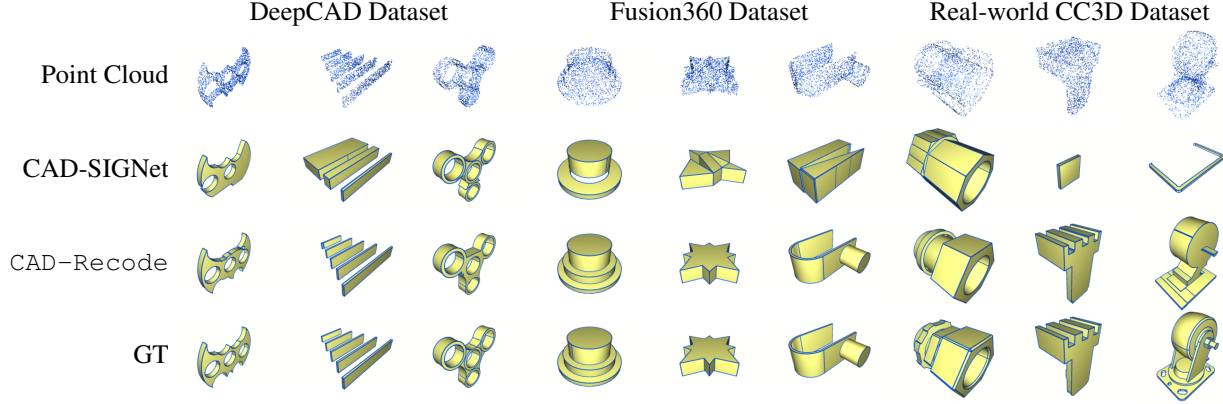


Figure 5. Qualitative results on the DeepCAD, Fusion360, and CC3D datasets. For each input point cloud (first row), we compare CAD models produced by CAD-SIGNet (second) and our CAD-Recode trained on our dataset (third) with a ground truth CAD model (bottom row). While CAD-SIGNet often fails to restore the general shape, CAD-Recode outputs only slightly deviate from ground truth in most cases.

Method	Mean CD↓	Med. CD↓	IoU↑	IR↓
DeepCAD [46]	–	263	–	12.7
CAD-SIGNet [21]	14.82	2.90	42.6	2.5
CAD-Recode	0.76	0.31	74.2	0.3

Table 2. Results on the CC3D dataset, where input point clouds are sampled from real 3D scans. CAD-Recode significantly outperforms DeepCAD, and CAD-SIGNet.

geometries that are much closer to the ground truth than current state-of-the-art. However, it can be observed that CAD-Recode still lacks the expressiveness to model complex shapes that contain operations beyond the extrusion operation such as revolution and fillet. This can be attributed to the choice of features and design practices in the procedurally generated training dataset. Nevertheless, we believe that this can be addressed in future works by incorporating further features in the dataset generation procedure. Our results on CC3D are compared with methods previously reported for this dataset [21], namely CAD-SIGNet and DeepCAD.

Ablation Study: To evaluate the different components of CAD-Recode, we conduct a comprehensive ablation study on the amount of training data, test-time sampling, and the number of input points and model parameters.

Training CAD-Recode on 160 k procedurally generated samples using the method described in Section 3.2 leads to significant improvements in geometric fidelity of the predicted samples over training on the DeepCAD dataset with the same amount of data (see row 2 and 3 of Table 3). Furthermore, scaling our training dataset to 1 M samples provides further improvements across all datasets (row 4 of Table 3). As compared to DeepCAD training dataset, our procedural dataset generation provides a better way of learning

the mapping between point clouds and CAD codes which can be further improved by scaling up the dataset size.

We investigate the effectiveness of the test-time sampling approach that generates multiple CAD code predictions through different point cloud samplings, as described in Section 4.3. As shown in the third and last row of Table 3, the test-time sampling approach mainly helps reducing the ratio of invalid predicted CAD codes (IR). For comparison, CAD-SIGNet [21] employs a probability-based sampling. Yet, even without test-time sampling our method still performs better on the reconstruction metrics than CAD-SIGNet [21].

Results in Table 4 show an ablation of the input point cloud size and the number of parameters of the LLM backbone. It can be observed that using an input point cloud of 256 points and Qwen1.5b results in the highest IoU. This setting with a relatively small input point cloud and lightweight LLM backbone provides the best balance between prediction accuracy and memory requirements. Results on all metrics are included in the supplementary materials.

5.2. CAD-QA and Editability

CAD-QA and LLM Interpretability: CAD SGP-Bench [37] is a benchmark of 1000 CAD-specific Question Answering (CAD-QA) tasks that test LLMs’ understanding of CAD model geometry from sketch-extrude sequences encoded as in DeepCAD [46]. These questions require analyzing various geometric aspects, such as relative sizes and 3D primitive types. We extend this benchmark to evaluate CAD reverse engineering methods by using point clouds as input instead of CAD sequences. Our evaluation follows a two-stage process: first predicting sketch-extrude sequences from point clouds as CadQuery code with CAD-Recode, then using GPT-4o [36] to an-

Method	Train Dataset		Test-time Sampling	DeepCAD			Fusion360			Real-World CC3D		
	Name	Size		CD↓	IoU↑	IR↓	CD↓	IoU↑	IR↓	CD↓	IoU↑	IR↓
Previous best [21]	DeepCAD	160 k	✓	3.43	77.6	0.9	7.37	65.6	1.6	14.80	42.6	4.4
CAD-Recode	DeepCAD	160 k	✓	0.89	86.2	0.0	1.77	75.6	0.0	2.43	62.5	0.0
CAD-Recode	Ours	160 k	✓	0.54	88.3	0.3	0.66	82.0	0.1	1.27	69.0	0.2
CAD-Recode	Ours	1 M	✓	0.30	92.0	0.4	0.35	87.8	0.5	0.76	74.2	0.3
Previous best [21]	DeepCAD	160 k	✗	6.81	77.3	4.4	14.5	58.4	9.3	32.59	39.1	15.5
CAD-Recode	Ours	1 M	✗	0.83	87.1	3.1	1.21	79.1	5.0	3.21	60.5	9.8

Table 3. Ablation of training data and test-time sampling. The results demonstrate the advantage of training on our procedurally generated data, while the test-time sampling helps reducing the invalidity ratio. CD stands for mean Chamfer distance.

Points	Model Size	DeepCAD		Fusion360		CC3D	
		CD↓	IoU↑	CD↓	IoU↑	CD↓	IoU↑
128	0.5 B	0.18	89.9	0.18	84.3	0.38	71.9
256	0.5 B	0.17	90.6	0.17	85.4	0.36	72.6
256	1.5 B	0.16	92.0	0.15	87.8	0.31	74.2

Table 4. Ablation of architecture details. CD stands for median Chamfer distance.

Method	CAD-QA Accuracy↑
PointLLM [47]	42.3
CAD-SIGNet [21] → GPT-4o	63.2
CAD-Recode → GPT-4o	76.5

Table 5. Point cloud CAD-specific question answering (CAD-QA) on the SGP-Bench benchmark. Our CAD-Recode supplied with a GPT-4o significantly outperforms baseline methods.

swer CAD-specific questions. Without requiring additional interpretation hints, our approach achieves 76.5% accuracy on this CAD-QA task (Table 5). For comparison, we evaluate two baseline approaches: CAD-SIGNet [21] and PointLLM [47]. When using CAD-SIGNet’s output with GPT-4o, even with provided interpretation hints explaining the sequence format, the accuracy reaches only 63.2%. PointLLM, which directly processes point clouds for language tasks, achieves 42.3% accuracy when prompted with the CAD-specific questions. These results demonstrate that CAD-Recode effectively captures CAD geometric information while generating an output in a format that proprietary LLMs can naturally interpret and process.

Editing Pipeline: Leveraging the interpretable nature of our code-based output, we present an automated editing pipeline using GPT-4o [36]. Through a simple prompt, the system refactors the generated code to expose geometric parameters via interactive sliders, enabling direct manipulation of the reconstructed model. As shown in Figure 6, the resulting code can be directly executed in a Python environ-

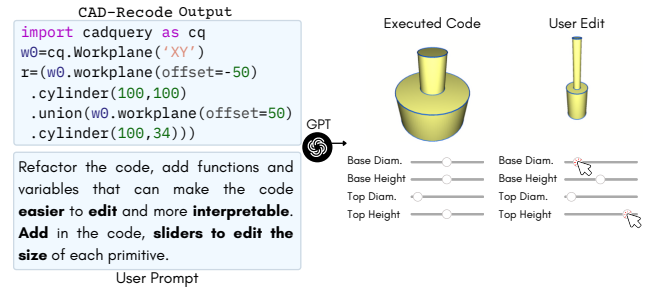


Figure 6. Interactive editing of a CAD model. Given the code output from CAD-Recode and a generic prompt, GPT-4o allows automated and interactive editing of the CAD model.

ment to provide an interactive editing interface. Implementation details are provided in the supplementary materials.

6. Conclusion

This work rethinks the problem of feature-based CAD reverse engineering by approaching it through pre-trained LLMs taking advantage of CAD Python-based representation. Our key contributions include a new CAD code representation for reverse engineering sketch-extrude sequences, very large-scale procedurally generated training dataset in the form of CAD programs, and a point cloud-to-CAD code model. We demonstrate that CAD-Recode outperforms existing methods by a large margin on three datasets, including the real-world CC3D dataset. We also showcase that combining pre-trained LLMs with geometric understanding enables powerful new workflows, where designers can reconstruct CAD models from point clouds and modify them through natural language. We believe that this work will open new perspectives for CAD reverse engineering. We identify the following interesting future works: (1) further exploiting the modularity of the proposed CAD code representation, (2) scaling up the LLM and the dataset to enable reverse engineering of more complex CAD models.

Acknowledgement: The present project is supported by the National Research Fund, Luxembourg under the BRIDGES2021/IS/16849599/FREE-3D and IF/17052459/CASCADES projects, and by Artec 3D.

References

- [1] Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Haider Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. Generating CAD code with vision-language models for 3d designs. In *ICLR*, 2025. 2
- [2] Akshay Badagabettu, Sai Sravan Yarlagadda, and Amir Barati Farimani. Query2cad: Generating cad models using natural language queries. *arXiv preprint arXiv:2406.00144*, 2024. 2
- [3] Antoine Brière-Côté, Louis Rivest, and Roland Maranzana. Comparing 3d cad models: uses, methods, tools and perspectives. *Computer-Aided Design and Applications*, 9(6): 771–794, 2012. 1
- [4] Jorge D Camba, Manuel Contero, and Pedro Company. Parametric cad modeling: An analysis of strategies for design reusability. *Computer-Aided Design*, 74:18–31, 2016. 1
- [5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021. 4
- [6] Sijin Chen, Xin Chen, Chi Zhang, Mingsheng Li, Gang Yu, Hao Fei, Hongyuan Zhu, Jiayuan Fan, and Tao Chen. Ll3da: Visual interactive instruction tuning for omni-3d understanding reasoning and planning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26428–26438, 2024. 2
- [7] Kseniya Cherenkova, Djamila Aouada, and Gleb Gusev. Pvdeconv: Point-voxel deconvolution for autoencoding cad construction in 3d. In *ICIP*, pages 2741–2745, 2020. 2, 4
- [8] Kseniya Cherenkova, Elona Dupont, Anis Kacem, Ilya Arzhannikov, Gleb Gusev, and Djamila Aouada. Sepicnet: Sharp edges recovery by parametric inference of curves in 3d shapes. In *CVPRW*, pages 2726–2734, 2023. 3
- [9] Kseniya Cherenkova, Elona Dupont, Anis Kacem, Gleb Gusev, and Djamila Aouada. Spelsnet: Surface primitive elements segmentation by b-rep graph structure supervision. *NeurIPS*, 2024. 3
- [10] CADQuery Developers. Cadquery: A python parametric cad scripting framework. <https://cadquery.readthedocs.io/>, 2024. Accessed: 2024-10-22. 2, 3, 4
- [11] Elona Dupont, Kseniya Cherenkova, Anis Kacem, Sk Aziz Ali, Ilya Aryhannikov, Gleb Gusev, and Djamila Aouada. Cadops-net: Jointly learning cad operation types and steps from boundary-representations. *3DV*, 2022. 3
- [12] Elona Dupont, Kseniya Cherenkova, Dimitrios Mallis, Gleb Gusev, Anis Kacem, and Djamila Aouada. Transcad: A hierarchical transformer for cad sequence inference from point clouds. In *Eur. Conf. Comput. Vis.*, 2024. 2, 3, 4, 6
- [13] David W Eggert, Andrew W Fitzgibbon, and Robert B Fisher. Simultaneous registration of multiple range views for use in reverse engineering of cad models. *Computer Vision and Image Understanding*, 69(3):253–272, 1998. 1
- [14] Markus Friedrich, Pierre-Alain Fayolle, Thomas Gabor, and Claudia Linnhoff-Popien. Optimizing evolutionary csg tree extraction. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 1183–1191, 2019. 3
- [15] Haoxiang Guo, Shilin Liu, Hao Pan, Yang Liu, Xin Tong, and Baining Guo. Complexgen: Cad reconstruction by b-rep chain complex generation. *ACM TOG*, 41(4):1–18, 2022. 3
- [16] Jiaming Han, Kaixiong Gong, Yiyuan Zhang, Jiaqi Wang, Kaipeng Zhang, Dahua Lin, Yu Qiao, Peng Gao, and Xiangyu Yue. Onellm: One framework to align all modalities with language. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26584–26595, 2024. 2
- [17] Robin H Helle and Hirpa G Lemu. A case study on use of 3d scanning for reverse engineering and quality control. *Materials Today: Proceedings*, 45:5255–5262, 2021. 1
- [18] Yining Hong, Haoyu Zhen, Peihao Chen, Shuhong Zheng, Yilun Du, Zhenfang Chen, and Chuang Gan. 3d-llm: Injecting the 3d world into large language models. *NeurIPS*, 2023. 2
- [19] Yining Hong, Haoyu Zhen, Peihao Chen, Shuhong Zheng, Yilun Du, Zhenfang Chen, and Chuang Gan. 3d-llm: Injecting the 3d world into large language models. *Advances in Neural Information Processing Systems*, 36:20482–20494, 2023. 2
- [20] Kacper Kania, Maciej Zieba, and Tomasz Kajdanowicz. Ucs-g-net-unsupervised discovering of constructive solid geometry tree. *Adv. Neural Inform. Process. Syst.*, 2020. 3
- [21] Mohammad Sadil Khan, Elona Dupont, Sk Aziz Ali, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-signet: Cad language inference from point clouds using layer-wise sketch instance guided attention. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 4713–4722, 2024. 1, 2, 3, 4, 5, 6, 7, 8
- [22] Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 9601–9611, 2019. 2, 4
- [23] Milin Kodnongbua, Benjamin Jones, Maaz Bin Safeer Ahmad, Vladimir Kim, and Adriana Schulz. Reparamcad: Zero-shot cad re-parameterization for interactive manipulation. In *SIGGRAPH Asia 2023 Conference Papers*, New

- York, NY, USA, 2023. Association for Computing Machinery. 2
- [24] Joseph G Lambourne, Karl DD Willis, Pradeep Kumar Jayaraman, Aditya Sanghi, Peter Meltzer, and Hooman Shayani. Brepnet: A topological message passing system for solid models. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 12773–12782, 2021. 3, 5
 - [25] Joseph George Lambourne, Karl Willis, Pradeep Kumar Jayaraman, Longfei Zhang, Aditya Sanghi, and Kamal Rahimi Malekshan. Reconstructing editable prismatic cad from rounded voxel models. In *SIGGRAPH Asia*, pages 1–9, 2022. 3, 6
 - [26] Lingxiao Li, Minhyuk Sung, Anastasia Dubrovina, Li Yi, and Leonidas J Guibas. Supervised fitting of geometric primitives to 3d point clouds. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 2652–2660, 2019. 3
 - [27] Ming Li, Frank C Langbein, and Ralph R Martin. Detecting design intent in approximate cad models using symmetry. *Computer-Aided Design*, 42(3):183–201, 2010. 1
 - [28] Pu Li, Jianwei Guo, Xiaopeng Zhang, and Dong-Ming Yan. Secad-net: Self-supervised cad reconstruction by learning sketch-extrude operations. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 16816–16826, 2023. 2, 3
 - [29] Pu Li, Jianwei Guo, Huibin Li, Bedrich Benes, and Dong-Ming Yan. Sfmcad: Unsupervised cad reconstruction by learning sketch-based feature modeling operations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4671–4680, 2024. 3
 - [30] Fei Liu. Fast industrial product design method and its application based on 3d cad system. *Computer-Aided Design and Applications*, 18(S3):118–128, 2020. 1
 - [31] Minghua Liu, Ruoxi Shi, Kaiming Kuang, Yinhao Zhu, Xuanlin Li, Shizhong Han, Hong Cai, Fatih Porikli, and Hao Su. Openshape: Scaling up 3d shape representation towards open-world understanding. *Advances in neural information processing systems*, 36, 2024. 2
 - [32] Yujia Liu, Stefano D’Aronco, Konrad Schindler, and Jan Dirk Wegner. Pc2wf: 3d wireframe reconstruction from raw point clouds. *ICLR*, 2021. 3
 - [33] Weijian Ma, Minyang Xu, Xueyang Li, and Xiangdong Zhou. Multicad: Contrastive representation learning for multi-modal 3d computer-aided design models. In *CIKM*, New York, NY, USA, 2023. Association for Computing Machinery. 2, 3, 4, 6
 - [34] Weijian Ma, Shuaiqi Chen, Yunzhong Lou, Xueyang Li, and Xiangdong Zhou. Draw step by step: Reconstructing cad construction sequences from point clouds via multimodal diffusion. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 27154–27163, 2024. 1, 2, 3, 4, 6
 - [35] Dimitrios Mallis, Ali Sk Aziz, Elona Dupont, Kseniya Cherenkova, Ahmet Serdar Karadeniz, Mohammad Sadil Khan, Anis Kacem, Gleb Gusev, and Djamila Aouada. Sharp challenge 2023: Solving cad history and parameters recovery from point clouds and 3d scans. overview, datasets, metrics, and baselines. In *CVPRW*, 2023. 6
 - [36] OpenAI. Gpt-4 technical report, 2024. 2, 3, 4, 6, 7, 8
 - [37] Zeju Qiu, Weiyang Liu, Haiwen Feng, Zhen Liu, Tim Z Xiao, Katherine M Collins, Joshua B Tenenbaum, Adrian Weller, Michael J Black, and Bernhard Schölkopf. Can large language models understand symbolic graphics programs? *arXiv preprint arXiv:2408.08313*, 2024. 2, 7
 - [38] Daxuan Ren, Jianmin Zheng, Jianfei Cai, Jiatong Li, and Junzhe Zhang. Extrudenet: Unsupervised inverse sketch-and-extrude for shape parsing. In *Eur. Conf. Comput. Vis.*, pages 482–498. Springer, 2022. 2, 3
 - [39] V Sanh. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019. 3
 - [40] Ari Seff, Wenda Zhou, Nick Richardson, and Ryan P Adams. Vitruvion: A generative model of parametric cad sketches. In *ICLR*, 2022. 3
 - [41] Gopal Sharma, Difan Liu, Subhansu Maji, Evangelos Kalogerakis, Siddhartha Chaudhuri, and Radomír Měch. Parsenet: A parametric surface fitting network for 3d point clouds. In *Eur. Conf. Comput. Vis.*, pages 261–276. Springer, 2020. 3
 - [42] Tarek M Sobh, J Owen, C Jaynes, Mohamed Dekhil, and Thomas C Henderson. Industrial inspection and reverse engineering. In *Proceedings of 1994 IEEE 2nd CAD-Based Vision Workshop*, pages 228–235. IEEE, 1994. 1
 - [43] Mikaela Angelina Uy, Yen-Yu Chang, Minhyuk Sung, Purvi Goel, Joseph G Lambourne, Tolga Birdal, and Leonidas J Guibas. Point2cyl: Reverse engineering 3d objects from point clouds to extrusion cylinders. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 11850–11860, 2022. 1, 2, 3, 6
 - [44] Xiaogang Wang, Yuelang Xu, Kai Xu, Andrea Tagliasacchi, Bin Zhou, Ali Mahdavi-Amiri, and Hao Zhang. Pie-net: Parametric inference of point cloud edges. *Adv. Neural Inform. Process. Syst.*, 33:20167–20178, 2020. 3
 - [45] Karl DD Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences. *ACM TOG*, 40(4):1–24, 2021. 2, 3, 4, 6
 - [46] Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 6772–6782, 2021. 2, 3, 4, 6, 7
 - [47] Runsen Xu, Xiaolong Wang, Tai Wang, Yilun Chen, Jiangmiao Pang, and Dahua Lin. Pointllm: Empowering large language models to understand point clouds. In *ECCV*, 2024. 2, 8
 - [48] Xianghao Xu, Wenzhe Peng, Chin-Yi Cheng, Karl DD Willis, and Daniel Ritchie. Inferring cad modeling sequences using zone graphs. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 6062–6070, 2021. 2, 3
 - [49] Xiang Xu, Karl DD Willis, Joseph G Lambourne, Chin-Yi Cheng, Pradeep Kumar Jayaraman, and Yasutaka Furukawa. Skexgen: Autoregressive generation of cad construction sequences with disentangled codebooks. In *ICML*, pages 24698–24724. PMLR, 2022. 3
 - [50] Xiang Xu, Pradeep Kumar Jayaraman, Joseph G Lambourne, Karl DD Willis, and Yasutaka Furukawa. Hierarchical neural coding for controllable cad model generation. *ICML*, 2023. 3, 6

- [51] Le Xue, Mingfei Gao, Chen Xing, Roberto Martín-Martín, Jiajun Wu, Caiming Xiong, Ran Xu, Juan Carlos Niebles, and Silvio Savarese. Ulip: Learning a unified representation of language, images, and point clouds for 3d understanding. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 1179–1189, 2023. [2](#)
- [52] Le Xue, Ning Yu, Shu Zhang, Artemis Panagopoulou, Junnan Li, Roberto Martín-Martín, Jiajun Wu, Caiming Xiong, Ran Xu, Juan Carlos Niebles, and Silvio Savarese. Ulip-2: Towards scalable multimodal pre-training for 3d understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 27091–27101, 2024. [2](#)
- [53] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report, 2024. [2](#), [4](#), [5](#)
- [54] Fukun Yin, Xin Chen, Chi Zhang, Biao Jiang, Zibo Zhao, Jiayuan Fan, Gang Yu, Taihao Li, and Tao Chen. Shapegpt: 3d shape generation with a unified multi-modal language model, 2023. [2](#)
- [55] Yang You, Mikaela Angelina Uy, Jiaqi Han, Rahul Thomas, Haotong Zhang, Suyu You, and Leonidas Guibas. Img2cad: Reverse engineering 3d cad models from images through vlm-assisted conditional factorization. *arXiv preprint arXiv:2408.01437*, 2024. [3](#)
- [56] Fenggen Yu, Qimin Chen, Maham Tanveer, Ali Mahdavi Amiri, and Hao Zhang. D2csg: Unsupervised learning of compact csg trees with dual complements and dropouts. *Advances in Neural Information Processing Systems*, 36: 22807–22819, 2023. [3](#)
- [57] Fenggen Yu, Qimin Chen, Maham Tanveer, Ali Mahdavi Amiri, and Hao Zhang. D2csg: Unsupervised learning of compact csg trees with dual complements and dropouts. *Advances in Neural Information Processing Systems*, 36, 2024. [3](#)
- [58] Haocheng Yuan, Jing Xu, Hao Pan, Adrien Bousseau, Niloy J Mitra, and Changjian Li. Cadtalk: An algorithm and benchmark for semantic commenting of cad programs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3753–3762, 2024. [2](#)
- [59] Renrui Zhang, Ziyu Guo, Wei Zhang, Kunchang Li, Xupeng Miao, Bin Cui, Yu Qiao, Peng Gao, and Hongsheng Li. Pointclip: Point cloud understanding by clip. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8552–8562, 2022. [2](#)
- [60] Zibo Zhao, Wen Liu, Xin Chen, Xianfang Zeng, Rui Wang, Pei Cheng, BIN FU, Tao Chen, Gang YU, and Shenghua Gao. Michelangelo: Conditional 3d shape generation based on shape-image-text aligned latent representation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [2](#), [5](#)
- [61] Xiangyu Zhu, Dong Du, Weikai Chen, Zhiyou Zhao, Yinyu Nie, and Xiaoguang Han. Nerve: Neural volumetric edges for parametric curve extraction from point cloud. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 13601–13610, 2023. [3](#)
- [62] Xiangyang Zhu, Renrui Zhang, Bowei He, Ziyu Guo, Ziyao Zeng, Zipeng Qin, Shanghang Zhang, and Peng Gao. Pointclip v2: Prompting clip and gpt for powerful 3d open-world learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2639–2650, 2023. [2](#)